

Q1
RAG & Eval

Q2
Agents & MCP

Q3
Fine-Tuning

Q4
Portfolio

12-Month AI Engineer

Learning Plan 2026

Senior Software Developer -> Senior AI / ML Engineer

4 quarters | 48+ weeks | Validated against mid-2026 market

Cloud: Google Cloud Platform (GCP)

Production RAG

LangGraph Agents

LLM Fine-tuning

MCP Servers

MLOps

GCP Certified

Background: Node.js | TypeScript | React | 10 years exp

Target: Senior AI Engineer | Roles relevant for 2026-2029

[*] 2026 – 2027

[*] Validated against mi...

[*] 63 tools

[*] 4 quarters

[*] 48 weeks

Advanced RAG + Eval infra

AI Assistant + MCP server

Fine-tuning + MLOps

Portfolio + Cert + Job

Background

[*] Node.js

[*] TypeScript

[*] React

[*] 10 years experience

[*] GenAI exposure

[*] RAG fundamentals

[*] Target role: Senior ...

[*] Relevant for 2026–2029

How to use this plan

This document is your week-by-week execution guide for the full 12-month journey from senior software developer to senior AI engineer. Each week has a clear theme, concrete tasks, tools to install and use, and a primary learning resource.

The four-quarter story

Each quarter builds directly on the previous one -- this is not four separate projects. It is one connected story:

- Q1 builds retrieval infrastructure and evaluation CI that Q2's assistant will run on top of.
- Q2 generates a golden Q&A dataset from the AI assistant that Q3 uses as fine-tuning training data.
- Q3 produces a fine-tuned model and MLOps pipeline that becomes the anchor portfolio piece in Q4.
- Q4 packages all three projects into a live, demoable portfolio and converts it into a job.

Weekly rhythm

- Each week has a primary theme. Stick to the theme -- do not jump ahead.
- Tasks marked with a star (★) indicate newly-added tools discovered from mid-2026 market research.
- Resources are one primary link per week. Do not collect 10 links -- go deep on one.
- End each week with a short GitHub commit or notes entry. Small, consistent outputs compound over 12 months.

LinkedIn parallel track (starts week 1)

The fastest career ROI comes from one short technical post per month describing what you built that month. By Q4 you will have 9 posts and inbound interest before you even apply anywhere. Write it in plain language -- what the problem was, what you built, what you learned.

Notation

- Tools in code font: standard tool already in the plan.
- Tools in green + bold: newly added from mid-2026 market research -- prioritise these.
- Resources listed as "Primary:" are the single most important thing to read/watch that week.

Goal: Build production-grade retrieval infrastructure and an automated evaluation CI pipeline. The output of Q1 becomes the backbone that the Q2 AI assistant runs on top of.

Deliverables at end of Q1:

- Hybrid retrieval pipeline (BM25 + dense vector + cross-encoder reranker) deployed on GCP
- Automated eval CI: RAGAS + DeepEval gates that block merge on metric regression
- Red-team test suite using Promptfoo -- adversarial prompts, prompt injection, jailbreak tests

- Three live deployment targets: Cloud Run (persistent), Cloud Functions (serverless), Ollama VPS (edge)
- Observability dashboard in Langfuse: latency, token cost, retrieval hit rate, failure modes

Month 1 -- Advanced RAG Patterns

You already know RAG fundamentals. This month goes deep on what separates production RAG from demos: hybrid retrieval combining BM25 keyword search with dense vector search, cross-encoder reranking, multi-vector ColBERT retrieval, and self-correcting RAG loops.

WEEK 1

Hybrid search architecture

Goals

- Understand why single-vector retrieval fails in production (vocabulary mismatch, exact-match gaps)
- Implement BM25 sparse retrieval using `pg_search` alongside `pgvector` dense retrieval
- Combine sparse + dense scores with Reciprocal Rank Fusion (RRF)

Weekly Tasks

- Set up a Postgres instance with both `pgvector` and `pg_search` extensions installed
- Index a document corpus with both a dense embedding column and a full-text tsvector column
- Implement an RRF fusion function that merges the two ranked lists
- Benchmark: run 20 queries, compare dense-only vs hybrid recall@10

Tools

[DB] pgvector[*] pg_search (BM25)[LI] LlamaIndex[API] FastAPI

Learning Resources

Primary: [Pinecone blog -- 'What is Hybrid Search?' \(covers BM25 + dense + RRF in depth\)](#). Secondary: [pgvector GitHub README -- full-text search + vectors section](#).

WEEK 1b

Chunking strategies and embedding model selection

Goals

- Understand why fixed-size chunking loses context at sentence boundaries -- compare 5 strategies
- Select the right embedding model for a code+text corpus using the MTEB leaderboard
- Measure how much chunking strategy and embedding choice move recall@10 independently

Weekly Tasks

- Implement and compare: fixed-size (512 tokens), sentence-window, recursive character, semantic (embedding similarity threshold), and AST-aware chunking on the same 500-document corpus
- Open the MTEB leaderboard (huggingface.co/spaces/mteb/leaderboard), filter by Retrieval task and models under 500M parameters -- shortlist 3 candidates
- Benchmark all 3 embedding models x 5 chunking strategies: run your 20-query set, measure recall@10 for each combination
- Pick the winning combination and hard-code it as the pipeline default for all subsequent Q1 weeks
- Log `chunking_strategy` and `embedding_model` as W&B run parameters so every experiment is reproducible

Tools

[*] Chonkie (fast Python... [*] LlamaIndex SentenceS... [ST] sentence-transformers
 [*] MTEB leaderboard [*] W&B [DB] pgvector

Learning Resources

Primary: Greg Kamradt's 'Chunking Strategies' notebook -- 5 methods benchmarked with recall scores (github.com/gkamradt/langchain-tutorials). Secondary: MTEB paper -- 'MTEB: Massive Text Embedding Benchmark' (Muennighoff et al. 2023) -- skim the Retrieval task section.

WEEK 2

Cross-encoder reranking

Goals

- Understand the two-stage retrieval architecture: broad recall (bi-encoder) then precise reranking (cross-encoder)
- Compare Cohere Rerank API (managed) vs BGE Reranker (self-hosted) on cost and latency
- Measure the NDCG@10 lift from adding reranking to the Week 1 hybrid pipeline

Weekly Tasks

- Integrate Cohere Rerank API into the retrieval pipeline as a post-retrieval step
- Also run BGE Reranker locally using HuggingFace sentence-transformers
- A/B test both rerankers on your 20-query benchmark set
- Log latency added vs metric improvement for each option

Tools

[*] Cohere Rerank [BGR] BGE Reranker [ST] sentence-transformers [DB] pgvector
 [SRCH] pg_search

Learning Resources

Primary: Cohere 'Reranking' documentation. Secondary: BAAI/bge-reranker-base model card on HuggingFace.

WEEK 3

ColBERT and multi-vector retrieval

Goals

- Understand late-interaction retrieval: why token-level embeddings beat single-vector for complex queries
- Run ColBERT/PLAID pipeline using RAGatouille wrapper
- Compare ColBERT retrieval quality vs bi-encoder on your benchmark

Weekly Tasks

- Install RAGatouille and index a 5000-document corpus with ColBERT
- Run retrieval queries and compare token-level MaxSim scores vs cosine similarity
- Test ColBERT on multi-hop questions where single-vector retrieval typically fails
- Profile memory and index size: ColBERT stores 128-dim vector per token, not per doc

Tools

[CB] ColBERT [RAG] RAGatouille [*] Qdrant (alternative ... [*] PyLATE

Learning Resources

Primary: ColBERT paper (Khattab & Zaharia 2020) -- skim Sections 1-3. Practical: RAGatouille GitHub README and notebook examples.

WEEK 4**Query routing and CRAG self-correcting loop****Goals**

- Build a query classifier that routes queries to the right retrieval strategy
- Implement Corrective RAG (CRAG): retrieve -> grade documents -> fall back to web search if low confidence
- Integrate DSPy for programmatic prompt optimisation (replaces manual prompt tuning)

Weekly Tasks

- Build a router: factual queries -> hybrid search; code queries -> ColBERT; conversational -> direct LLM
- Implement a relevance grader (LLM-as-judge scoring 0-1 on each retrieved chunk)
- Add fallback path: if mean relevance < 0.5, escalate to Tavily web search
- Try DSPy to auto-optimize the grader prompt rather than hand-tuning it

Tools

[*] LangChain (routing)

[LI] LlamaIndex

[DSP] DSPy

[*] Instructor (structur...

[TAV] Tavily API

Learning Resources

Primary: CRAG paper -- 'Corrective Retrieval Augmented Generation' (2024). DSPy: Getting Started tutorial on dspy.ai.

Month 1b -- Data Engineering for AI

Real AI engineers spend 30-40% of their time on data -- not model code. This mini-month covers the unglamorous work that determines whether your RAG pipeline works in production: ingesting messy documents, deduplicating, scrubbing PII, and keeping the index fresh as source data changes. These tasks are probed heavily in Senior AI Engineer interviews.

WEEK 1c**Document ingestion, deduplication, and incremental indexing****Goals**

- Ingest documents from multiple messy real-world formats: PDFs (scanned and digital), HTML pages, DOCX files, and Markdown
- Detect and remove near-duplicate chunks before indexing using MinHash LSH
- Implement incremental upsert: only re-embed and re-index documents that changed since the last run
- Scrub PII from text before it enters the vector store

Weekly Tasks

- Use Unstructured.io to load PDFs (digital + scanned via OCR), HTML, DOCX, and Markdown into a unified document list
- Intentionally introduce 50 near-duplicate documents; use datasketch MinHash LSH to detect pairs with Jaccard similarity > 0.85 -- confirm they are removed before indexing
- Add a content hash column to your pgvector table; on re-index, only upsert rows where hash differs from stored value -- measure speedup on a 5000-doc corpus
- Run a simple regex + spaCy NER pass to detect and redact emails, phone numbers, and Israeli ID numbers before embedding
- Log deduplication rate and PII hit rate as W&B metrics -- these tell you about data quality over time

Tools

[*] Unstructured.io

[*] datasketch (MinHash ...

[*] spaCy (NER for PII)

[DB] pgvector

[*] W&B

[*] LlamaIndex SimpleDir...

Learning Resources

Primary: *Unstructured.io* documentation -- 'Supported File Types' and 'Partitioning' sections. Secondary: 'Deduplicating Training Data Makes Language Models Better' (Lee et al. 2022) -- skim Section 2 for MinHash intuition.

Month 2 -- LLM Evaluation Deep Dive

Evaluation is what separates production AI from demos. This month you build a CI pipeline that measures retrieval and generation quality on every commit and blocks merge if metrics regress. You also add red-teaming and adversarial tests.

WEEK 5

RAGAS -- RAG-specific metrics

Goals

- Understand what each of the 6 RAGAS metrics measures and when each can fail
- Run RAGAS on the Q1 pipeline with a ground-truth dataset of 50 question-answer-context triples
- Understand the difference between reference-free metrics (faithfulness) and reference-based (answer correctness)

Weekly Tasks

- Install RAGAS and create a 50-sample evaluation dataset (hand-label 10, generate 40 synthetically with GPT-4o)
- Run the full metric suite: Faithfulness, Answer Relevancy, Context Precision, Context Recall, Answer Similarity, Answer Correctness
- Log results to W&B as a baseline experiment
- Identify the weakest metric -- that is where to improve next

Tools

[EVAL] RAGAS

[*] W&B

[*] GPT-4o (synthetic da...

[DB] pgvector

[API] FastAPI

Learning Resources

Primary: *RAGAS documentation* -- 'Metrics' section (ragas.io/docs). Secondary: *RAGAS paper* -- skim the metric definitions in Section 3.

WEEK 6

DeepEval and CI eval gates

Goals

- Integrate DeepEval into a Pytest test suite so evals run on every git push
- Set score thresholds that block merge if metrics drop below baseline
- Understand LLM-as-judge: using GPT-4o or Claude as the evaluator of answer quality

Weekly Tasks

- Install DeepEval and convert the RAGAS dataset into DeepEval test cases
- Write Pytest tests for: Faithfulness ≥ 0.8 , Answer Relevancy ≥ 0.75 , Context Recall ≥ 0.7
- Push a deliberately bad change (retrieve wrong chunks) and confirm tests block the commit
- Add a GitHub Actions workflow: on push -> run eval suite -> fail if thresholds not met

Tools

[TEST] DeepEval**[PY]** Pytest**[GHA]** GitHub Actions**[*]** GPT-4o (judge)**[*]** W&B**Learning Resources**

Primary: DeepEval documentation -- 'Getting Started' and 'Metrics' sections (docs.confident-ai.com). Secondary: DeepEval GitHub README.

WEEK 7**Red teaming and adversarial tests with Promptfoo****Goals**

- Understand the attack surface of a RAG system: prompt injection, jailbreaks, data extraction, context poisoning
- Run Promptfoo to systematically test adversarial inputs against the pipeline
- Add a security eval stage to the CI pipeline

Weekly Tasks

- Install Promptfoo and configure a test suite against your FastAPI RAG endpoint
- Test categories: direct prompt injection ('ignore previous instructions'), indirect injection (malicious text in retrieved chunks), PII extraction attempts, jailbreaks
- Generate a Promptfoo HTML report and identify the top 3 vulnerabilities
- Patch the most critical issue (typically: system prompt leak or context poisoning)

Tools**[RED]** Promptfoo**[API]** FastAPI**[GHA]** GitHub Actions**[*]** LangSmith (trace adv...)**Learning Resources**

Primary: Promptfoo documentation -- 'Red Team' section (promptfoo.dev/docs/red-team). Secondary: OWASP Top 10 for LLMs (2025 edition).

WEEK 8**Braintrust and automated release gating****Goals**

- Set up Braintrust as a CI-integrated evaluation platform with score summaries and merge blocking
- Understand the difference between Braintrust (release gate), LangSmith (trace explorer), and Langfuse (OSS self-hosted observability)
- Configure Langfuse self-hosted as the production observability layer

Weekly Tasks

- Sign up for Braintrust and push the RAGAS dataset as a Braintrust experiment
- Configure GitHub Actions: Braintrust CLI runs on PR, posts score delta as a PR comment
- Spin up Langfuse via Docker Compose on a local or cloud VM
- Instrument the FastAPI RAG service with Langfuse SDK: trace every request, log token cost, log retrieval hit rate

Tools**[CI]** Braintrust**[OBS]** Langfuse**[*]** OTel GenAI (tracing ...)**[LS]** LangSmith**[*]** Arize Phoenix**[DOC]** Docker**Learning Resources**

Primary: Langfuse self-hosting guide (langfuse.com/docs/deployment/self-host). Secondary: Braintrust 'Evals as CI' guide.

Month 3 -- Multi-Target Deployment

With the RAG pipeline and eval CI in place, this month focuses on deploying to three target environments: a persistent containerised service on Google Cloud Run, a serverless endpoint on Google Cloud Functions, and an edge deployment on a VPS using Ollama. You also wire up production monitoring.

WEEK 9

Google Cloud Run containerised deployment

Goals

- Containerise the FastAPI RAG service with Docker and push to Google Artifact Registry
- Deploy to Cloud Run with auto-scaling and a Cloud Load Balancer
- Understand Cloud Run service configuration, traffic splitting, and rolling deployments

Weekly Tasks

- Write a production Dockerfile for the FastAPI service (multi-stage, non-root user, health check endpoint)
- Push image to Google Artifact Registry
- Deploy the container to Cloud Run: set min-instances=1, max-instances=10, CPU 60% target
- Set up a Cloud Load Balancer with a Cloud Armor security policy in front of Cloud Run
- Run the eval CI suite against the Cloud Run endpoint to confirm parity with local

Tools

[DOC] Docker

[AR] Google Artifact Regi...

[CR] Cloud Run

[LB] Cloud Load Balancing

[API] FastAPI

[GHA] GitHub Actions

Learning Resources

Primary: Google Cloud Run documentation -- 'Deploying container images' and 'Configuring auto-scaling'. Secondary: Docker Best Practices for Python applications.

WEEK 10

Google Cloud Functions serverless inference

Goals

- Deploy a lightweight RAG endpoint as a Cloud Functions (2nd gen) function using container images
- Understand Cloud Functions cold-start tradeoffs for LLM workloads and mitigation strategies (min-instances)
- Wire Cloud Functions to Cloud API Gateway for an HTTP endpoint

Weekly Tasks

- Package the RAG service as a Cloud Functions container image (Artifact Registry -> Cloud Functions)
- Set min-instances=1 on Cloud Functions to eliminate cold starts on the first request
- Set up Cloud API Gateway in front of Cloud Functions
- Run latency benchmarks: Cloud Run vs Cloud Functions (p50, p95, p99) -- document the cold-start delta
- Decide threshold: use Lambda when < 50 req/day, ECS when > 200 req/day

Tools

[CF] Cloud Functions**[GW]** Cloud API Gateway**[DOC]** Docker**[AR]** Google Artifact Regi...**[MON]** Cloud Monitoring**Learning Resources**

Primary: Google Cloud Functions (2nd gen) documentation -- 'Deploy a container image'. Secondary: Cloud Functions cold-start mitigation -- Google blog post on min-instances.

WEEK 11**Ollama VPS edge deployment****Goals**

- Deploy an open-source model (Llama 3.2 or Mistral) on a cheap VPS using Ollama
- Understand the use case: offline capability, data privacy, cost elimination for high-volume inference
- Set up Nginx reverse proxy with basic auth in front of Ollama

Weekly Tasks

- Provision a VPS (Hetzner CX21 or similar, ~€5/month) and install Ollama
- Pull Llama-3.2-3B-Instruct (fits in 2 GB RAM) and verify generation via curl
- Add Nginx with basic auth and HTTPS (Let's Encrypt)
- Swap the LLM client in the FastAPI service to point at the Ollama endpoint
- Run the RAGAS eval suite against the Ollama-backed service and compare scores vs cloud model

Tools**[OLL]** Ollama**[NGX]** Nginx**[API]** FastAPI**[EVAL]** RAGAS**[*]** Let's Encrypt**Learning Resources**

Primary: Ollama GitHub README -- 'REST API' section. Secondary: Nginx basic auth setup guide.

WEEK 12**Q1 wrap-up: production polish and LinkedIn post****Goals**

- Connect all observability: every deployment target sends traces to Langfuse
- Document the architecture with a README and architecture diagram
- Write and publish the first LinkedIn post: what you built, the hybrid search architecture, the eval CI approach

Weekly Tasks

- Add Langfuse tracing to both Cloud Run and Cloud Functions deployments (not just local)
- Set up Langfuse alerts: error rate > 2% sends an email notification
- Write a concise GitHub README with: architecture diagram, RAGAS benchmark table, deployment instructions
- Post on LinkedIn: 500-800 words, focus on the problem (why single-vector RAG fails) and the solution

Tools**[OBS]** Langfuse**[*]** OTel GenAI**[GH]** GitHub**[EVAL]** RAGAS**[TEST]** DeepEval**Learning Resources**

Primary: Langfuse 'Monitoring' documentation. Resource for LinkedIn post: study how Anthropic and Weaviate write technical blog posts -- problem-first, then solution.

Goal: Build a production-grade AI assistant for the XSScan codebase that you can query before writing any feature, ask architectural questions, and have multi-turn conversations with. The assistant uses a 3-index design (code AST + rules + domain docs) and is exposed as an MCP server on npm so it works in Claude Code and Cursor with a single config line.

Note: If XSScan is early-stage when Q2 starts, use a well-structured open-source TypeScript/Node.js project as the corpus temporarily, then migrate to XSScan once it has enough code surface.

Deliverables at end of Q2:

- 3-index retrieval system: code index (Tree-sitter AST symbols), rules index (conventions), domain index (docs + tickets)
- Query classifier that routes each question to the right index
- Multi-turn conversation memory with LangGraph checkpointing
- TypeScript MCP server published on npm, live in Claude Code + Cursor via one config line
- Golden Q&A dataset of 200+ pairs -- feeds Q3 fine-tuning directly
- Langfuse dashboard: query latency, index hit rates, conversation quality scores

Month 4 -- LangGraph Stateful Agents

LangGraph is the production standard for stateful multi-agent workflows. This month you build the conversation backbone the assistant will use: stateful graphs, supervisor pattern for routing, human-in-the-loop checkpoints, and persistent conversation memory.

WEEK 13

LangGraph fundamentals

Goals

- Understand the LangGraph mental model: nodes (functions), edges (transitions), state (typed dict), and the graph compile step
- Build a minimal stateful conversation agent with memory that persists across turns
- Understand the difference between LangGraph (stateful graphs) and raw LangChain (chain composition)

Weekly Tasks

- Install LangGraph and build a 3-node graph: receive query -> retrieve context -> generate answer
- Add a MemorySaver checkpointer so conversation history persists across turns
- Test multi-turn: ask a follow-up question that references the previous answer -- confirm the agent uses context
- Instrument with LangSmith: view the full trace, token counts, and latency per node

Tools

GRAPH LangGraph

[*] Vercel AI SDK (TypeS...

[LS] LangSmith

[*] Python

Learning Resources

Primary: LangGraph documentation -- 'Quickstart: Build a Chatbot' tutorial. Secondary: LangGraph GitHub -- examples/memory-chatbot.

WEEK 14

Supervisor pattern and multi-agent routing

Goals

- Implement the supervisor pattern: a top-level LLM decides which sub-agent to call based on query type
- Build 3 sub-agents: code-search agent, rules-lookup agent, domain-docs agent
- Understand human-in-the-loop: interrupt the graph before a tool call for user approval

Weekly Tasks

- Build a supervisor node that classifies queries into: code / rules / domain
- Each sub-agent calls a different retrieval function (stubs for now -- real indexes wired in Month 5)
- Add a human-in-the-loop interrupt: when confidence < 0.6, ask the user for clarification before proceeding
- Test with 10 different query types and verify routing accuracy

Tools

GRAPH LangGraph
 [V] Vercel AI SDK
 [*] Pydantic AI (type-sa...)
 [*] LiteLLM (provider ab...)
 [LS] LangSmith

Learning Resources

Primary: LangGraph documentation -- 'Multi-Agent' section. Secondary: LangGraph 'Human-in-the-loop' how-to guide.

WEEK 15

Vercel AI SDK and TypeScript agent layer

Goals

- Understand why Vercel AI SDK is the TypeScript-native choice for AI applications in 2026
- Build a TypeScript wrapper around the Python LangGraph backend using Vercel AI SDK
- Implement streaming responses so the IDE shows text as it generates, not after

Weekly Tasks

- Install Vercel AI SDK and configure providers: Anthropic Claude Sonnet as primary, fallback to OpenAI GPT-4o via LiteLLM
- Build a TypeScript API route that calls the Python LangGraph service and streams the response
- Implement tool calling in the TypeScript layer: the SDK handles tool definitions, invocations, and results automatically
- Test streaming in a simple terminal client

Tools

[V] Vercel AI SDK
 [LIT] LiteLLM
 [*] TypeScript
 [*] Node.js
 GRAPH LangGraph

Learning Resources

Primary: Vercel AI SDK documentation -- 'Getting Started' and 'Streaming' sections (sdk.vercel.ai). Secondary: 'Tool Use' section of the same docs.

WEEK 16

CrewAI multi-agent teams

Goals

- Understand role-based multi-agent design: Researcher, Analyst, Responder roles working in sequence
- Build a CrewAI pipeline for the assistant: one agent retrieves, one grades relevance, one synthesises the answer
- Compare CrewAI (role-based) vs LangGraph (graph-based): know when to use each

Weekly Tasks

- Build a 3-agent CrewAI crew: Retriever agent, Relevance Grader agent, Answer Synthesiser agent
- Each agent has a specific system prompt defining its role and tools
- Run a set of 20 complex queries through CrewAI and through the LangGraph agent -- compare answer quality and latency
- Decision: for the assistant, use LangGraph (more control) with CrewAI patterns (role clarity)

Tools

[CRW] CrewAI

[GRAPH] LangGraph

[V] Vercel AI SDK

[*] Langfuse (trace both)

Learning Resources

Primary: CrewAI documentation -- 'Core Concepts' (docs.crewai.com). Secondary: CrewAI examples -- 'research-and-write' example.

Month 5 -- 3-Index Design and Codebase Parsing

The core intellectual challenge of Q2: how do you make an AI assistant that genuinely understands a large codebase? The answer is three separate indexes, each optimised for a different type of question, with a query classifier routing to the right one.

WEEK 17**Tree-sitter AST parsing for the code index****Goals**

- Use Tree-sitter to extract structured symbols from TypeScript/JavaScript source files
- Extract: function names and signatures, class definitions, import/export relationships, call graphs
- Understand why line-based chunking fails for code (splits functions mid-body) vs AST-based chunking

Weekly Tasks

- Install tree-sitter and tree-sitter-typescript Python bindings
- Write a parser that walks the XSScan AST and extracts all exported functions, classes, and their docstrings
- Build a call graph: for each function, record which functions it calls
- Output a JSON file: { file, symbol, type, signature, docstring, calls[] } for every symbol
- Manually inspect 20 entries to verify accuracy

Tools

[AST] Tree-sitter

[*] tree-sitter-typescript

[*] Python

[*] JSON

Learning Resources

Primary: Tree-sitter documentation -- 'Using Parsers' section (tree-sitter.github.io). Secondary: tree-sitter-typescript GitHub -- grammar.js for TypeScript node types.

WEEK 18**Building and querying the 3-index system****Goals**

- Build all three indexes: code index (Tree-sitter symbols), rules index (CLAUDE.md + convention files), domain index (docs + tickets)
- Implement the query classifier: classify each incoming query as code / rules / domain
- Wire classifier output to the correct retrieval function in the LangGraph supervisor

Weekly Tasks

- Index code symbols into pgvector with embeddings of signature + docstring
- Index rules and conventions as chunks in Qdrant (use Qdrant's payload filters to tag by rule file)
- Index domain docs (README, architecture docs, Jira ticket descriptions) in a third collection
- Build the query classifier using a small prompted LLM call (Claude Haiku -- fast and cheap)
- Test: ask 'what does X function do' (code), 'how should I format dates' (rules), 'what is the payment flow' (domain)

Tools

[DB] pgvector

[VDB] Qdrant

[AST] Tree-sitter

[GRAPH] LangGraph

[V] Vercel AI SDK

[*] Instructor (typed cl...

Learning Resources

Primary: Qdrant documentation -- 'Payload Filtering' section (qdrant.tech/documentation). Secondary: Anthropic prompt caching guide (for caching system prompts with large rule sets).

WEEK 18b**AI safety, guardrails, and responsible AI in practice****Goals**

- Understand output guardrails: input validation, output filtering, and topic blocking as engineering controls
- Know the EU AI Act risk tiers well enough to classify your own assistant (likely Limited Risk -- transparency obligation)
- Implement NeMo Guardrails on the FastAPI RAG endpoint with 3 active rails: topic, jailbreak, and output format

Weekly Tasks

- Study Constitutional AI basics (Anthropic 2022): understand how values are embedded during RLHF alignment -- this context makes guardrails design decisions make sense
- Read the OWASP Top 10 for LLMs 2025 edition -- map each vulnerability to a concrete mitigation already in the pipeline (Week 7 covers several, identify the gaps)
- Install NVIDIA NeMo Guardrails and configure 3 rails on the RAG FastAPI endpoint: (1) topic rail -- block off-topic questions, (2) jailbreak detection rail, (3) output format rail -- enforce JSON schema on structured endpoints
- Test each rail: send 10 violating inputs per rail, verify all 10 are blocked; send 20 legitimate inputs, verify 0 are blocked (false positive rate)
- Write a one-page Responsible AI policy for the XSScan assistant: intended use, prohibited use, data handling, human oversight mechanism

Tools

[NMO] NeMo Guardrails

[*] Guardrails for Amazo...

[*] Promptfoo (extend re...

[API] FastAPI

Learning Resources

Primary: NVIDIA NeMo Guardrails documentation -- 'Getting Started' and 'Topical Rails' sections (docs.nvidia.com/nemo/guardrails). Secondary: Anthropic Constitutional AI paper (2022) -- Sections 1-3 for alignment context. Also: EU AI Act summary -- Article 52 transparency obligations for AI systems.

WEEK 19**Conversation memory and context window management****Goals**

- Implement sliding window memory: keep the last N turns, summarise older turns
- Handle context window limits: a large codebase index + conversation history can exceed the context window
- Implement entity memory: remember key entities (function names, files) mentioned in the conversation

Weekly Tasks

- Add a summarisation step that fires when conversation history > 8000 tokens: summarise older turns into 2-3 sentences
- Add entity extraction: after each user message, extract mentioned code symbols and store them as conversation state

- Use those entities to bias retrieval in subsequent turns (entity-aware re-query)
- Test a 10-turn conversation that starts broad and narrows to a specific function

Tools

[*] LangGraph (MemorySav...

[V] Vercel AI SDK

[API] FastAPI

[OBS] Langfuse

Learning Resources

Primary: LangGraph 'Memory' documentation -- how-to guides. Secondary: Anthropic 'Long context' cookbook (context window management strategies).

WEEK 20

Golden Q&A dataset generation

Goals

- Generate 200+ high-quality Q&A pairs from the XSScan codebase -- this is the Q3 training data
- Ensure diversity: code questions, rules questions, domain questions, multi-hop questions
- Quality-filter synthetically generated pairs before including them in the dataset

Weekly Tasks

- Write a GPT-4o prompt that generates Q&A pairs given a code chunk or document chunk as context
- Generate 300 pairs across all three index types
- Run each pair through the assistant and score answer quality with RAGAS Answer Similarity metric
- Keep pairs where RAGAS score ≥ 0.75 : target at least 200 after filtering
- Store final dataset in JSONL format: { question, answer, context, index_type, source_file }

Tools

[*] GPT-4o (generation)

[*] RAGAS (quality filter)

[*] Instructor (typed ge...

[GH] GitHub

Learning Resources

Primary: RAGAS documentation -- 'Synthetic Test Set Generation' section. Secondary: Anthropic cookbook -- 'Generating synthetic evaluation datasets'.

Month 6 -- MCP Server and IDE Integration

This month takes the Python LangGraph assistant and wraps it in a TypeScript MCP server so developers can query it directly from Claude Code and Cursor. The MCP server is published on npm as an installable package.

WEEK 21

MCP server fundamentals

Goals

- Understand the MCP protocol: tools, resources, prompts, and the client-server architecture
- Build a minimal TypeScript MCP server with one tool using @modelcontextprotocol/sdk
- Test it with MCP Inspector (the Postman equivalent for MCP) before connecting to any client

Weekly Tasks

- Install @modelcontextprotocol/sdk and scaffold a minimal MCP server

- Define one tool: `ask_codebase(question: string) -> calls the FastAPI assistant endpoint`
- Run `npx @modelcontextprotocol/inspector` to test the tool call -- verify request/response in the browser UI
- Connect to Claude Desktop: add to `claude_desktop_config.json`, restart, test in conversation

Tools

[*] @modelcontextprotoco...

[*] MCP Inspector

[*] TypeScript

[*] Node.js

Learning Resources

Primary: MCP TypeScript SDK documentation (modelcontextprotocol.io/docs/sdk). Secondary: MCP Inspector README (github.com/modelcontextprotocol/inspector).

WEEK 22

Full MCP server with resources and HTTP transport

Goals

- Add resource endpoints to the MCP server: expose the index stats and recent queries as readable resources
- Switch from stdio transport (single-client) to `StreamableHttpServerTransport` (multi-client, remote)
- Use Hono as the lightweight HTTP server layer

Weekly Tasks

- Add 3 tools: `ask_codebase`, `list_indexed_files`, `search_rules(topic: string)`
- Add 2 resources: `index_stats` (file count, symbol count, last indexed) and `recent_queries` (last 10 questions + answers)
- Switch transport to `StreamableHttpServerTransport` with Hono handling the HTTP routing
- Deploy the MCP server to a VPS or Railway alongside the FastAPI backend
- Test all tools via MCP Inspector against the remote HTTP transport

Tools

[*] @modelcontextprotoco...

[*] MCP Inspector

[HON] Hono

[*] TypeScript

[*] Railway (deploy)

Learning Resources

Primary: MCP documentation -- 'Transports' section. Secondary: Hono documentation -- 'Getting Started' (hono.dev).

WEEK 23

npm publish and Claude Code + Cursor integration

Goals

- Package the MCP server for distribution on npm
- Test installation via `npx`: the server should start with zero configuration beyond an API key
- Wire into Claude Code and Cursor -- one config entry, working tools

Weekly Tasks

- Add a bin entry to `package.json` so the server starts via `npx xsscan-assistant`
- Write a `schema.json` with tool definitions, descriptions, and input schemas
- Publish to npm (public package, version 0.1.0)
- Add to Claude Code: `.claude/mcp.json` with the `npx` command
- Add to Cursor: `settings.json` `mcpServers` entry
- Test end-to-end: ask a codebase question in Claude Code -- verify it hits the real assistant

Tools

[NPM] npm

[*] @modelcontextprotoco...

[*] Claude Code

[*] Cursor

[*] MCP Inspector

Learning Resources

Primary: [npm publishing guide](#) -- 'Creating and publishing unscoped packages'. Secondary: [Claude Code MCP documentation](#).

WEEK 24**Q2 wrap-up: observability and LinkedIn post****Goals**

- Wire Langfuse into every path: Python FastAPI backend + TypeScript MCP server
- Create a Langfuse dashboard showing: daily queries, index hit rates, answer quality trend, cost per query
- Write and publish LinkedIn post #2: the MCP server -- how it works, what it solves, how to install it

Weekly Tasks

- Instrument MCP tool calls in Langfuse: each tool call creates a span with input, output, latency
- Set up a weekly digest: Langfuse sends a summary email on Mondays (total queries, top questions, cost)
- Write the GitHub README with: 1-minute install GIF, architecture diagram, golden Q&A examples
- Publish on npm and announce on LinkedIn: 800-1000 words, demo GIF, link to npm package

Tools

[OBS] Langfuse

[V] Vercel AI SDK

[GH] GitHub

[NPM] npm

[*] MCP Inspector

Learning Resources

Primary: [Langfuse documentation](#) -- 'Integrations' section for SDK instrumentation.

Month 6b -- Python ML Fundamentals (Pre-Q3 Bridge)

You are arriving at fine-tuning from a TypeScript/React background. Q3 training code is NumPy, Pandas, PyTorch tensors, and HuggingFace datasets. Without a deliberate two-week ramp, you will copy-paste training scripts without understanding them -- and interviewers will find this gap quickly. This bridge month closes it before you touch a GPU.

WEEK 6b.1**NumPy, Pandas, and PyTorch tensor fundamentals****Goals**

- Operate fluently on PyTorch tensors: shape inspection, reshaping, broadcasting, device movement (CPU <-> GPU)
- Use Pandas and NumPy for data wrangling: loading CSVs/JSONL, filtering, groupby, apply, and vectorised operations
- Read and interpret a training loop without needing to look anything up

Weekly Tasks

- Complete fast.ai Practical Deep Learning Lesson 1-2 (free, [fastai.fast.ai](#)) -- covers PyTorch tensor operations in a practical context
- Write a training loop from scratch: load data, forward pass, compute loss, backward pass, optimizer step -- do this without copy-pasting, line by line

- Load the Q2 golden Q&A JSONL dataset using Pandas: filter by `index_type`, compute answer length statistics, plot a histogram of RAGAS scores -- practice data wrangling on real data from your own project
- Profile a simple forward pass: use `torch.profiler` to see where time is spent; understand what "CUDA kernel launch" means in the output
- Run inference on a HuggingFace model using the pipeline API, then run the same inference using the raw `model.generate()` call -- understand what the pipeline does for you

Tools

[PT] PyTorch [NP] NumPy [PD] Pandas [*] Matplotlib [*] HuggingFace transformers
[*] fast.ai (course) [*] Google Colab

Learning Resources

Primary: fast.ai 'Practical Deep Learning for Coders' Lessons 1-2 (course.fast.ai) -- highly practical, TypeScript-developer-friendly explanations. Secondary: PyTorch documentation -- 'Tensors' tutorial (pytorch.org/tutorials).

WEEK 6b.2

HuggingFace datasets, tokenizers, and training diagnostics

Goals

- Load, inspect, filter, and preprocess datasets using the HuggingFace datasets library
- Understand tokenization deeply: how text becomes token IDs, what special tokens do, how to handle overflow and truncation
- Read a training loss curve and diagnose overfitting, underfitting, and learning rate issues before wasting GPU budget

Weekly Tasks

- Load 3 different dataset formats using `datasets.load_dataset`: a CSV, a JSONL file (your Q2 golden dataset), and a HuggingFace Hub dataset
- Apply a tokenizer to a batch: inspect `input_ids`, `attention_mask`, and `token_type_ids` -- decode back to text to verify correctness
- Write a custom Dataset class in PyTorch that wraps your golden Q&A data and returns `(input_ids, labels)` tensors ready for a Trainer
- Run a 10-step training job and deliberately create 3 pathological loss curves: healthy descent, flat (LR too low), diverging (LR too high) -- screenshot all three for future reference
- Use W&B to log the 3 loss curves; practice reading the W&B Runs comparison view -- this is the exact interface you will use in Q3

Tools

[*] HuggingFace datasets [*] HuggingFace tokenizers [PT] PyTorch [*] W&B
[*] Google Colab [*] TRL (preview)

Learning Resources

Primary: HuggingFace 'NLP Course' Chapters 2-3 (huggingface.co/learn/nlp-course) -- tokenizers and fine-tuning. Secondary: W&B 'Intro to MLOps' course (free, wandb.ai/courses) -- focus on the experiment tracking module.

Goal: Take the Q2 golden Q&A dataset and fine-tune a base model using QLoRA, then improve alignment with DPO preference tuning. Build a reproducible MLOps pipeline: git push triggers training, eval, model registry entry, and serving. The fine-tuned model should measurably outperform the base model on the RAGAS benchmark.

Deliverables at end of Q3:

- SFT model (QLoRA) trained on the 200+ golden Q&A pairs -- RAGAS scores better than base model
- DPO-aligned model: preferred responses on code review and domain Q&A tasks

- Reproducible CI pipeline: git push -> train -> eval -> registry -> serve, end-to-end
- Pre/post RAGAS comparison report published as a GitHub Gist
- Model served via vLLM or HF Inference Endpoints -- live, queryable, demoable

Month 7 -- QLoRA Fine-Tuning

QLoRA (Quantised Low-Rank Adaptation) lets you fine-tune large models on a single consumer GPU or a cheap cloud GPU by combining 4-bit quantisation with LoRA adapters. Unsloth makes this 2-5x faster. This month you run your first SFT training job on the Q2 dataset.

WEEK 25

QLoRA fundamentals and environment setup

Goals

- Understand the QLoRA math: 4-bit NF4 quantisation + double quantisation + LoRA adapters trained in bfloat16
- Set up a GPU training environment: Google Colab A100 for experimentation, Modal for longer runs
- Understand bitsandbytes, PEFT, and how they interact with the Hugging Face Trainer

Weekly Tasks

- Open a Colab notebook with A100 runtime
- Install: bitsandbytes, PEFT, TRL, transformers, accelerate, unsloth
- Load Llama-3.2-3B-Instruct in 4-bit using BitsAndBytesConfig
- Verify: model loads in ~2 GB VRAM, dtype is nf4
- Run a test inference to confirm the model generates sensible text before training

Tools

[*] HF PEFT

[BNB] bitsandbytes

[TRL] TRL

[FAST] Unsloth

[*] Google Colab

[GPU] Modal

Learning Resources

Primary: Unsloth documentation -- 'Llama 3.2 fine-tuning' notebook. Secondary: QLoRA paper -- 'QLoRA: Efficient Finetuning of Quantized LLMs' (Dettmers et al. 2023) -- skim Sections 1-3.

WEEK 26

SFT training with TRL SFTTrainer + Unsloth

Goals

- Format the Q2 golden dataset in ChatML or Alpaca instruction format
- Train a LoRA adapter using TRL SFTTrainer with Unsloth's speed optimisations
- Understand the key hyperparameters: rank (r), alpha, target modules, batch size, gradient accumulation

Weekly Tasks

- Convert the 200 golden Q&A pairs to the instruction format: { instruction, input, output }
- Configure LoRA: r=16, alpha=32, target_modules=['q_proj','v_proj'], dropout=0.05
- Train for 3 epochs on Colab A100 -- should take ~20 minutes with Unsloth
- Save the LoRA adapter weights to HuggingFace Hub (private repo)
- Log training loss curve to W&B -- confirm loss decreases monotonically

Tools

[*] TRL SFTTrainer [FAST] Unsloth [*] HF PEFT [*] W&B [HFH] HuggingFace Hub

Learning Resources

Primary: TRL SFTTrainer documentation (huggingface.co/docs/trl/sft_trainer). Secondary: Unsloth 'SFT Notebook' on GitHub.

WEEK 27

Evaluation: base model vs fine-tuned model

Goals

- Run RAGAS evaluation on both the base model and the fine-tuned model on a held-out test set
- Understand what to expect: SFT improves format adherence and domain-specific accuracy; it does not replace pretraining
- Identify which metric improved most and which didn't move -- this tells you what DPO needs to fix

Weekly Tasks

- Reserve 20 Q&A pairs as a held-out test set (not used in training)
- Run both models on all 20 test questions
- Score both sets of answers with RAGAS: Faithfulness, Answer Relevancy, Answer Similarity
- Build a comparison table: metric | base model | fine-tuned model | delta
- Log everything to W&B as two experiments in the same project

Tools

[EVAL] RAGAS [*] W&B [HFH] HuggingFace Hub [TRL] TRL [FAST] Unsloth

Learning Resources

Primary: RAGAS documentation -- 'Evaluating LLMs' section. Secondary: W&B documentation -- 'Compare experiments' guide.

WEEK 28

Faster training with Modal and Axolotl

Goals

- Move training from Colab (session-based, interrupts) to Modal (on-demand GPU, persistent)
- Use Axolotl for config-driven training: replaces writing Python training scripts with a YAML config file
- Understand when to graduate from Unsloth (single GPU, easy) to Axolotl (multi-GPU, config-driven)

Weekly Tasks

- Write a Modal Python script that provisions an A100, runs Unsloth training, saves adapter to HF Hub, then tears down
- Confirm Modal run costs < \$2 for the 200-pair fine-tune
- Convert the training config to Axolotl YAML format -- run the same fine-tune via Axolotl
- Compare Axolotl vs Unsloth: training speed, memory usage, adapter quality (RAGAS score)

Tools

[GPU] Modal [AX] Axolotl [FAST] Unsloth [*] HF PEFT [*] W&B

Learning Resources

Primary: Axolotl documentation -- 'Getting Started' (github.com/axolotl-org/axolotl). Secondary: Modal documentation -- 'Running GPU workloads'.

Month 8 -- DPO Alignment and Preference Tuning

DPO (Direct Preference Optimisation) trains the model to prefer certain response styles over others using pairs of (chosen, rejected) responses. It is more stable than PPO/RLHF and requires no reward model. This month you build a preference dataset and run DPO on top of the SFT model.

WEEK 29

Building a preference dataset

Goals

- Understand the DPO data format: { prompt, chosen, rejected } where chosen is the preferred response
- Generate preference pairs using the assistant: ask the same question with different prompts, rate responses
- Understand what makes a response 'preferred' for the codebase assistant use case

Weekly Tasks

- For each of 50 training prompts, generate 3 candidate answers using different temperature settings
- Rank the 3 answers by: factual accuracy, conciseness, use of correct code identifiers from the codebase
- Highest-ranked = chosen, lowest-ranked = rejected
- Store in JSONL format: { prompt, chosen, rejected, source_index_type }
- Manually verify 10 pairs to confirm the ranking criteria are consistent

Tools

[*] GPT-4o (generation)

[*] Instructor (structur...

[FAST] Unsloth

[HFH] HuggingFace Hub

Learning Resources

Primary: DPO paper -- 'Direct Preference Optimization: Your Language Model is Secretly a Reward Model' (Rafailov et al. 2023) -- Sections 1-3. Secondary: TRL DPOTrainer documentation.

WEEK 30

DPO training

Goals

- Run DPO fine-tuning on top of the SFT model from Month 7
- Understand DPO loss: maximise likelihood of chosen response, minimise likelihood of rejected response, with KL penalty to stay close to reference model
- Monitor key DPO training metrics: rewards/chosen, rewards/rejected, reward margin

Weekly Tasks

- Load the SFT adapter and apply DPO training using TRL DPOTrainer
- Key hyperparameters: beta=0.1 (KL penalty coefficient), learning_rate=5e-5, num_train_epochs=2
- Monitor on W&B: reward margin should increase across training steps
- Save the DPO adapter to HF Hub alongside the SFT adapter
- Run the RAGAS eval on the DPO model -- compare: base -> SFT -> DPO

Tools

[*] TRL DPOTrainer

[AX] Axolotl

[FAST] Unsloth

[*] W&B

[HFH] HuggingFace Hub

Learning Resources

Primary: TRL DPOTrainer documentation (huggingface.co/docs/trl/dpo_trainer). Secondary: Axolotl DPO config examples on GitHub.

WEEK 31

GRPO for reasoning tasks

Goals

- Understand GRPO (Group Relative Policy Optimisation): uses multiple completions per prompt, ranks them, uses rank as reward signal
- GRPO is better than DPO for reasoning tasks (code generation, multi-step Q&A) because it uses group statistics
- Run a small GRPO experiment on code-related prompts

Weekly Tasks

- Generate 30 code-related prompts where the model needs to reason step-by-step (e.g., 'trace the call path from X to Y')
- Use TRL GRPOTrainer with a simple reward function: answer contains the correct function name AND is under 300 tokens
- Compare GRPO vs DPO on the code-question subset of the test set
- Document the tradeoff: GRPO is better for reasoning, DPO is better for style alignment

Tools

[*] TRL GRPOTrainer [AX] Axolotl [*] W&B [HFH] HuggingFace Hub

Learning Resources

Primary: TRL GRPOTrainer documentation. Secondary: 'DeepSeek-R1' paper -- Section 3 describes the GRPO setup used for reasoning.

WEEK 32

MLflow model registry and versioning

Goals

- Register the base, SFT, and DPO models in MLflow with metrics, parameters, and tags
- Understand the MLflow model lifecycle: Staging -> Production -> Archived
- Set up model comparison views in MLflow: side-by-side metric comparison across versions

Weekly Tasks

- Start an MLflow tracking server locally (mlflow server --host 0.0.0.0)
- Log all three model versions with: RAGAS scores, training parameters, adapter size, training time, cost
- Promote the DPO model to 'Production' stage in the registry
- Add tags: { dataset_version, training_date, base_model, fine_tune_type }
- Write a model card for the HuggingFace Hub entry: training data, eval results, limitations

Tools

[MLF] MLflow [*] W&B [HFH] HuggingFace Hub [AX] Axolotl

Learning Resources

Primary: MLflow documentation -- 'Model Registry' section (mlflow.org/docs). Secondary: HuggingFace model card specification.

Month 9 -- MLOps Pipeline and Model Serving

The fine-tuned model is valuable only if it's deployed and monitored. This month closes the loop: a full CI/CD pipeline where a git push triggers training, evaluation, registry entry, and deployment. You also set up vLLM and SGLang for production inference.

WEEK 33**vLLM production inference****Goals**

- Deploy the fine-tuned model using vLLM for production multi-user serving
- Understand PagedAttention: how vLLM achieves high throughput via GPU memory paging
- Understand the OpenAI-compatible API that vLLM exposes (drop-in replacement for the OpenAI client)

Weekly Tasks

- Install vLLM and serve the DPO adapter: `vllm serve <base_model> --enable-lora --lora-modules my-adapter=<adapter_path>`
- Hit the vLLM OpenAI-compatible endpoint with curl: confirm the adapted model responds correctly
- Run a throughput benchmark: 50 concurrent requests, measure tokens/second and P95 latency
- Compare vLLM vs Ollama on a 10-request batch: vLLM should be 5-10x faster on concurrent load

Tools**[vLLM]** vLLM**[*]** SGLang (benchmark ag...**[EVAL]** RAGAS**[API]** FastAPI**Learning Resources**

Primary: vLLM documentation -- 'Serving with LoRA' and 'OpenAI-Compatible Server' sections. Secondary: vLLM GitHub -- performance benchmarks.

WEEK 34**SGLang and prefix caching****Goals**

- Understand SGLang's RadixAttention: automatic prefix caching for shared system prompts across requests
- This is a major win for RAG/agent workloads where many requests share the same retrieved context chunks
- Benchmark SGLang vs vLLM on the codebase assistant workload (which has long shared system prompts)

Weekly Tasks

- Install SGLang and serve the same DPO adapter
- Design a benchmark that simulates the real assistant workload: 80% of requests share the first 2000 tokens (retrieved context)
- Run 100 requests through both vLLM and SGLang, measure tokens/second and TTFT (time to first token)
- Document the result: if SGLang is faster on this workload (it should be due to prefix caching), use it for production

Tools**[SGL]** SGLang**[vLLM]** vLLM**[*]** Python (benchmark sc...**Learning Resources**

Primary: SGLang documentation -- 'RadixAttention' section (docs.sglang.ai). Secondary: SGLang GitHub -- benchmark scripts.

WEEK 35**Full CI/CD training pipeline****Goals**

- Build a GitHub Actions pipeline: push to training branch -> run fine-tune on Modal -> run RAGAS eval -> register in MLflow -> deploy to vLLM
- The pipeline should be fully reproducible: same data, same config, same result every time
- Add a quality gate: only promote to production if RAGAS faithfulness ≥ 0.80

Weekly Tasks

- GitHub Actions workflow: trigger on push to training/*, steps: checkout -> Modal training job -> RAGAS eval script -> MLflow log -> conditional deploy
- Modal training job: provision A100, run Axolotl config, save adapter to HF Hub, return eval metrics
- Quality gate: if metrics pass threshold, trigger vLLM rolling deployment via gcloud run deploy command
- Test the full pipeline end-to-end twice to confirm reproducibility

Tools

[GHA] GitHub Actions [GPU] Modal [AX] Axolotl [MLF] MLflow [VLLM] vLLM [*] W&B
[EVAL] RAGAS [TEST] DeepEval

Learning Resources

Primary: GitHub Actions documentation -- 'Using secrets' and 'Environments with required reviewers'. Secondary: Modal documentation -- 'Running from GitHub Actions'.

WEEK 36

Q3 wrap-up: benchmark report and LinkedIn post

Goals

- Publish the pre/post RAGAS comparison report as a public GitHub Gist
- Deploy the model to HuggingFace Inference Endpoints for easy portfolio demo access
- Write and publish LinkedIn post #3: the fine-tuning journey -- QLoRA, DPO, and the metric results

Weekly Tasks

- Write the RAGAS comparison report: table of all metrics across base / SFT / DPO, with analysis of why each changed
- Deploy to HF Inference Endpoints (serverless, pay-per-request -- ideal for demos)
- Build a Gradio demo UI that queries the HF endpoint and streams the response
- Publish Gradio demo on HF Spaces (free hosting)
- LinkedIn post: share the RAGAS benchmark numbers, what worked, what surprised you

Tools

[*] HF Inference Endpoints [UI] Gradio [*] HF Spaces [GH] GitHub [EVAL] RAGAS
[*] W&B

Learning Resources

Primary: HuggingFace Inference Endpoints documentation. Secondary: Gradio documentation -- 'Quickstart'.

Goal: Convert 9 months of compounding work into a job. Three demoable portfolio projects, one GCP certification, and an active job search strategy targeting Senior AI Engineer and ML Engineer roles at companies using RAG and agents in production.

Deliverables at end of Q4:

- Three live, demoable portfolio projects with architecture diagrams and benchmark tables
- Google Cloud Professional Machine Learning Engineer (PME) certification

- LinkedIn profile rewritten as AI Engineer with 9 content posts published during the year
- 10+ target companies contacted with personalised outreach
- System design interview prep completed: at least 3 mock interviews on Pramp or Interviewing.io

Month 10 -- Portfolio Packaging

Raw GitHub repos are not a portfolio. This month you make each project tell a story: what problem it solves, what architecture it uses, what the measured outcomes are, and how to run it in 5 minutes. You add live demo UIs so interviewers can try the system without installing anything.

WEEK 37

Project 1 portfolio packaging: Production RAG service

Goals

- Write a README that leads with the problem (why production RAG is hard) not the solution
- Add a RAGAS benchmark table showing before/after hybrid search + reranker metrics
- Add an architecture diagram: data flow from query to retrieval to generation to response

Weekly Tasks

- Write README sections: Problem Statement, Architecture, Benchmarks, Quick Start (5 minutes to running), Key Decisions
- Create an architecture diagram using draw.io or Excalidraw -- export as SVG, embed in README
- Record a 2-minute demo GIF using Gifox or Loom: show a query going through the pipeline, the RAGAS metrics, and the Langfuse dashboard
- Add a GitHub Actions badge showing the latest eval CI run status

Tools

[GH] GitHub

[*] draw.io / Excalidraw

[OBS] Langfuse

[EVAL] RAGAS

Learning Resources

Primary: Good README examples to study: Weaviate, LlamaIndex, LangGraph GitHub READMEs. Notice: problem-first, architecture diagram, benchmark table, Quick Start.

WEEK 37b

Open-source contribution to a core AI library

Goals

- Land at least 1 merged PR to a well-known AI library -- this is a resume line and a signal no bootcamp graduate can fake
- Choose a contribution scope that is achievable in 1 week but genuinely useful (documentation fixes do not count)
- Use the contribution as LinkedIn post material and a portfolio talking point

Weekly Tasks

- Identify your target repo: LlamaIndex, LangGraph, RAGAS, DeepEval, or Unsloth -- pick the one you know best from Q1-Q3
- Search the repo's GitHub issues for labels: "good first issue", "help wanted", "bug"; -- shortlist 3 issues where you can make a real fix using your Q1-Q3 knowledge
- Implement the fix or improvement: write tests, follow the project's contribution guide, open a PR with a clear description of what the bug was and why your fix is correct
- Respond to reviewer feedback within 24 hours -- the speed of response is noticed and appreciated by maintainers
- Once merged: add the contribution to your LinkedIn Featured section and GitHub profile README with a 1-sentence description of the impact

Tools**[GH]** GitHub**[PY]** pytest**[*]** the target library's...**[*]** your Q1-Q3 project c...**Learning Resources**

Primary: 'How to Contribute to Open Source' guide (opensource.guide/how-to-contribute). Secondary: Watch 1-2 merged PRs in your target repo to understand the reviewer's expectations and the project's code style before opening your own.

WEEK 38**Project 2 portfolio packaging: XSScan AI assistant + MCP server****Goals**

- The MCP server is the most novel piece -- lead with that in the README
- Add a one-command install section: the reviewer should be able to add the assistant to their IDE in under 2 minutes
- Record a demo GIF showing a real conversation with the assistant from inside Claude Code

Weekly Tasks

- Write README sections: What it does, Architecture (3-index design diagram), One-command install, Example conversations, Benchmark (RAGAS on golden dataset)
- Record a 3-minute demo GIF: install the npm package, add to Claude Code config, restart, ask 3 real codebase questions -- show real answers
- Update the npm package to v1.0.0 with a proper package description and keywords for discoverability
- Link the npm package from the GitHub README and LinkedIn profile

Tools**[GH]** GitHub**[NPM]** npm**[*]** MCP Inspector**[*]** Claude Code**[*]** Cursor**Learning Resources**

Primary: Study the README of popular MCP servers on the MCP Registry -- observe what makes them immediately compelling.

WEEK 39**Project 3 portfolio packaging: Fine-tuned model + MLOps pipeline****Goals**

- The RAGAS benchmark comparison is the headline: base model vs SFT vs DPO in one table
- Add a CI pipeline diagram showing the full automation: push -> train -> eval -> registry -> serve
- Build a Gradio demo that lets the interviewer query the fine-tuned model vs the base model side-by-side

Weekly Tasks

- Write README sections: Why fine-tune, Training approach (QLoRA + DPO), Results (RAGAS table), Reproducibility (run this command to reproduce), Architecture
- Build a Gradio side-by-side demo: two text boxes, same question, base model left / fine-tuned model right
- Host on HuggingFace Spaces (free, no setup for the viewer)
- Write and publish the RAGAS comparison report as a Medium article (technical blog, 1000-2000 words)

Tools**[UI]** Gradio**[*]** HF Spaces**[*]** HF Inference Endpoints**[GH]** GitHub**[MLF]** MLflow**[*]** Medium

Learning Resources

Primary: Gradio documentation -- 'Blocks layout' for side-by-side comparison UI. Secondary: study how fast.ai blog posts are structured (problem, approach, results, lessons).

WEEK 39b**AI product thinking and metric design for interviews****Goals**

- Define success metrics for an AI feature from first principles -- not just RAGAS, but business outcomes
- Design a valid A/B test for an AI system where the treatment is non-deterministic
- Communicate model quality to a non-technical PM or stakeholder in 5 minutes without losing their attention

Weekly Tasks

- For each of your 3 portfolio projects, write a one-page product brief: problem, user, success metric, guardrail metric, how you would know if the feature is failing in production -- this forces product thinking on your own work
- Design an A/B test for the RAG assistant: what is the control, what is the treatment, what is the randomisation unit (user vs query), what is the minimum detectable effect, how long to run -- write this up as if presenting to a PM
- Practice the 'AI quality to non-technical stakeholder' conversation: explain RAGAS faithfulness score in 2 sentences without using the word 'embedding' -- record yourself on video, watch it back
- Study 3 AI product failure post-mortems (search 'AI product failure post-mortem' on Substack and Hacker News) -- for each one, identify which metric would have caught the failure earlier
- Prepare answers for the 3 most common AI product interview questions: 'How would you measure the success of this AI feature?', 'How do you decide when the model is good enough to ship?', 'What happens when the model is wrong?'

Tools

[*] Notion / Google Docs...

[*] Loom (video practice)

[*] Statsig or LaunchDarkly

Learning Resources

Primary: Shreya Shankar's blog post 'What I Wish Someone Told Me About Building AI Products' (eugeneyan.com has a curated list). Secondary: Chip Huyen 'AI Engineering' Chapter 1 -- framing AI problems as product decisions.

WEEK 40**LinkedIn and personal brand****Goals**

- Rewrite the LinkedIn profile as an AI Engineer, not a software developer
- Review all 9 LinkedIn posts from the past 9 months and pin the best one
- Connect with AI engineers at target companies -- warm outreach beats cold applications 10:1

Weekly Tasks

- Rewrite LinkedIn headline: 'Senior AI Engineer | Production RAG · LangGraph Agents · LLM Fine-tuning'
- Rewrite About section: 3 paragraphs -- who you are, what you've built, what you're looking for
- Add portfolio section with links to all 3 GitHub repos and the HF Spaces demos
- Identify 20 target companies (startups using RAG/agents in prod -- use Wellfound + LinkedIn search)
- Send 10 personalised connection requests with a 2-sentence note referencing something specific about their AI work

Tools

[*] LinkedIn

[*] Wellfound

[GH] GitHub

[*] HF Spaces

Learning Resources

Primary: Study LinkedIn profiles of Senior AI Engineers at companies like Cohere, Weaviate, Weights & Biases, Anthropic. Notice the headline format and About section structure.

Month 11 -- Google Cloud Professional ML Engineer Certification

The Google Cloud Professional Machine Learning Engineer (PME) certification covers ML pipeline design, model training, deployment, monitoring, and Responsible AI on GCP. 4-6 weeks of prep, one sitting. It directly validates the Vertex AI, Cloud Run, and MLOps skills you built in Q1-Q3.

WEEK 40b**Resume rewrite with quantified AI outcomes****Goals**

- Rewrite your resume as an AI Engineer with quantified outcomes from the 3 portfolio projects
- Understand the difference between your LinkedIn profile (discoverable narrative) and your resume (interview screening document)
- Pass ATS screening for "Senior AI Engineer" roles at target companies

Weekly Tasks

- Rewrite the resume header: name, email, GitHub, LinkedIn, HuggingFace profile, personal site (if any) -- no photo, no address
- Rewrite the Summary section in 3 bullets: (1) what you build (RAG + agents + fine-tuning), (2) your stack (Python, TypeScript, LangGraph, GCP), (3) what you have shipped (3 production projects, 1 GCP cert, 1 npm package with X downloads)
- For each portfolio project, write 3 bullet points with the format: "Built [what] using [how] achieving [measured outcome]" -- e.g. "Implemented hybrid BM25 + dense retrieval pipeline achieving 34% recall@10 improvement over single-vector baseline on a 5,000-document corpus"
- Rewrite your current job experience: extract any AI-adjacent work (automation, data pipelines, API integrations) and reframe it toward AI engineering vocabulary
- Run your resume through an ATS simulator (Jobscan or Resume Worded) against 3 Senior AI Engineer job descriptions -- fix keyword gaps before sending

Tools

[*] Jobscan or Resume Wo...

[*] Google Docs or LaTeX...

[*] LinkedIn (cross-refe...

Learning Resources

Primary: Teal HQ blog -- 'How to Write an AI Engineer Resume' (practical, with examples). Secondary: Study 5 LinkedIn profiles of Senior AI Engineers who recently changed jobs -- notice how they quantify their project outcomes.

WEEK 41**Google Cloud AI/ML services overview****Goals**

- Map the Google Cloud AI/ML service landscape: which services replace which traditional ML steps
- Understand Vertex AI (foundation models + full ML platform), Model Garden (pre-trained APIs), and AI APIs (Vision AI, Natural Language AI, Document AI)

- Understand the PME exam structure: 60 questions, 2 hours, scored pass/fail -- covers ML problem framing, data prep, model development, pipelines, and monitoring

Weekly Tasks

- Work through Google Cloud Skills Boost: 'Machine Learning on Google Cloud' learning path (free with trial credits)
- Summarise each GCP AI service in one sentence: what it does, when you'd use it vs build your own
- Take the Google Cloud Skills Boost PME pre-assessment to identify weak areas
- Focus on Section 3 (ML pipelines on Vertex AI) and Section 5 (Model monitoring and maintenance) first -- highest exam weight

Tools

[GCP] Google Cloud Skills ...

[VAI] Vertex AI

[MG] Model Garden

Learning Resources

Primary: Google Cloud Skills Boost -- 'Preparing for the Google Cloud Professional Machine Learning Engineer Exam' course. Secondary: Google Cloud documentation -- Vertex AI overview.

WEEK 42

Responsible AI and Google Cloud AI governance

Goals

- Understand the Responsible AI pillars covered in AIF-C01: fairness, explainability, privacy, security, transparency, governance
- Know the Google Responsible AI tools: Vertex Explainable AI (feature attribution), Model Cards, Vertex AI Model Armor (guardrails)
- Understand AI governance frameworks: EU AI Act basics, NIST AI RMF

Weekly Tasks

- Study the PME exam section on responsible AI practices -- this is often underestimated and has high per-question weight
- Read Google's Responsible AI Practices guide (ai.google/responsibility/responsible-ai-practices)
- Practice with 20 Responsible AI scenario questions from Google Cloud Skills Boost practice questions
- Build a glossary of Responsible AI terms with definitions in your own words

Tools

[GCP] Google Cloud Skills ...

[XAI] Vertex Explainable AI

[ARM] Vertex AI Model Armor

Learning Resources

Primary: Google's Responsible AI Practices guide (aws.amazon.com/machine-learning/responsible-machine-learning). Secondary: NIST AI Risk Management Framework overview.

WEEK 43

Exam practice and scheduling

Goals

- Complete 3 full practice exams under timed conditions
- Target score of 800+ on practice exams before sitting the real exam (pass = 700)
- Understand the question style: scenario-based, GCP-service focused, choose-the-best-approach

Weekly Tasks

- Take Whizlabs GCP ML Engineer practice exam set 1 -- score and review every wrong answer
- Take Tutorials Dojo GCP ML Engineer practice exam -- focus on the domain with the most misses

- Take Google Cloud Skills Boost official practice question set -- these are closest to the real exam
- Schedule the real exam for the end of week 44 or early week 45 (Pearson VUE, online proctored)

Tools

[GCP] Google Cloud Skills ...

[*] Whizlabs GCP ML Engi...

[*] Tutorials Dojo GCP M...

EXAM Kryterion

Learning Resources

Primary: Google Cloud Skills Boost -- Official Practice Question Set for Professional ML Engineer. Secondary: Tutorials Dojo GCP ML Engineer cheat sheets.

WEEK 44

Sit the exam + announce on LinkedIn

Goals

- Pass the Google Cloud Professional ML Engineer exam
- Announce the certification on LinkedIn -- this generates significant profile visibility
- Add the certification to your LinkedIn profile and resume

Weekly Tasks

- Sit the Professional ML Engineer exam (online proctored via Kryterion / webassessor.com/googlecloud)
- On pass: immediately announce on LinkedIn with the Google Cloud badge image + one paragraph on why GCP certification reinforces practical project work
- Add to LinkedIn certifications section, GitHub profile README, and resume
- Treat this as LinkedIn post #10 -- certification posts get high engagement

Tools

[*] Google Cloud Profess...

[*] LinkedIn

[*] Credly (Google Cloud...

Learning Resources

No study resource this week -- final review only. Sleep 8 hours before the exam.

Month 12 -- Active Job Search

Month 12 is execution month: applying, interviewing, and negotiating. The portfolio is live, the certification is done, and you have 9 months of documented public work. The job search strategy is targeted, not spray-and-pray.

WEEK 45

Target company list and application strategy

Goals

- Build a list of 30 target companies using AI/RAG/agents in production in 2026-2027
- Understand the difference between: AI-native startups (fastest learning, highest equity), scale-ups (balance), and enterprise AI teams (highest salary, slowest pace)
- Prioritise companies where you have a warm connection or where your MCP server / RAG work is directly relevant

Weekly Tasks

- Use Wellfound to filter: AI Engineer roles, B-D round funded, < 200 employees -- these are the fastest-moving teams

- Use LinkedIn to find companies that have recently posted about RAG, LangGraph, fine-tuning, or MCP
- Research each company's technical blog or GitHub to understand their AI stack before applying
- Write a tailored 3-sentence application note for each company: what you built, why it's relevant to them, what you want to work on

Tools

[*] Wellfound [*] LinkedIn [*] levels.fyi [*] GitHub (research)

Learning Resources

Primary: Lenny's Podcast episode on AI engineer hiring in 2025/2026. Secondary: Chip Huyen's blog -- 'How to get a job as an AI engineer'.

WEEK 46

System design interview preparation

Goals

- Study the top 5 LLM system design interview questions that appear at Senior AI Engineer level
- Practice explaining the full RAG architecture, fine-tuning pipeline, and multi-agent design in 10 minutes each
- Understand evaluation-driven design: interviewers want to know how you measure quality, not just how you build features

Weekly Tasks

- Study question: Design a production RAG system for a 1M-document corpus with SLA of < 2 seconds P95 latency
- Study question: How would you fine-tune an LLM for a specific domain with only 500 training examples?
- Study question: Design the observability layer for a multi-agent system in production
- Practice all three out loud, timing yourself to 10 minutes each
- Book a Pramp session for end of week 47

Tools

[*] Pramp [*] Interviewing.io [*] draw.io (whiteboard ...)

Learning Resources

Primary: Chip Huyen's 'Designing Machine Learning Systems' book -- Chapter 11 (ML System Design). Secondary: Alex Xu 'System Design Interview' Vol. 2 -- ML chapter.

WEEK 47

Mock interviews

Goals

- Complete at least 2 mock system design interviews with peers or on Pramp
- Get feedback and identify specific gaps in your explanations
- Practice the portfolio walkthrough: 5-minute demo of each project, then answer technical deep-dives

Weekly Tasks

- Pramp session 1: system design -- design a production RAG service
- Pramp session 2: ML design -- design a fine-tuning pipeline for a domain-specific chatbot
- Interviewing.io session: live technical interview with a working AI engineer
- Practice the portfolio walkthrough: screen-share all 3 projects, explain each in 3 minutes

Tools

[*] Pramp

[*] Interviewing.io

[*] GitHub (live demo)

[*] HF Spaces (live demo)

Learning Resources

Primary: Your own projects -- the best preparation is being able to answer every possible 'how does it work' and 'why did you choose X' question about code you wrote.

WEEK 48**Offers and negotiation****Goals**

- Understand how to evaluate AI Engineer offers: base salary, equity (type, vesting, cliff), AI budget (GPU credits), learning opportunities
- Understand how to negotiate: always negotiate, use competing offers, never give a number first
- Celebrate: you shipped 3 production-grade AI projects, got certified, and changed careers in 12 months

Weekly Tasks

- Research comp ranges on levels.fyi for Senior AI Engineer roles at your target companies
- Practice the negotiation conversation out loud: 'I'm very excited about this role. My target is X based on Y comparable offers'
- Evaluate offers on: technical challenge, team quality, equity upside, AI budget, remote flexibility
- Make the decision and sign

Tools

[*] levels.fyi

[*] LinkedIn (comp resea...

[*] Blind (anonymous com...

Learning Resources

Primary: Haseeb Qureshi's salary negotiation blog -- 'Ten Rules for Negotiating a Job Offer'. It's for software engineers but every principle applies.

Complete Tools Inventory

63 tools across all 4 quarters. Green bold = newly added from mid-2026 market research. Star (★) marks highest-priority new tools.

Q1 Tools -- Advanced RAG, Evaluation, Deployment

Category

Tools

Status

Vector databases

[DB] pgvector

[*] pg_search (BM25)

[VDB] Qdrant

[*] Weaviate

All production-stable

Reranking

[*] ColBERT / RAGatouille

[*] Cohere Rerank

[BGR] BGE Reranker

[*] PyLATE

Cross-encoder required in prod

RAG orchestration

[LI] LlamaIndex [DSP] DSPy [INS] Instructor [*] LangChain (ecosystem)

LangChain chains declining

Evaluation

[EVAL] RAGAS [TEST] DeepEval [RED] Promptfoo [CI] Braintrust

All stable / rising

Observability

[OBS] Langfuse [*] OTEL GenAI [*] Arize Phoenix [LS] LangSmith

Langfuse = OSS leader 2026

Deployment

[API] FastAPI [DOC] Docker [CF] Cloud Functions [CR] Cloud Run [OLL] Ollama
[GHA] GitHub Actions [*] W&B

All stable

Q2 Tools -- AI Assistant, LangGraph, MCP Server

Category

Tools

Status

Agent orchestration

[GRAPH] LangGraph [V] Vercel AI SDK [PYD] Pydantic AI [LIT] LiteLLM [CRW] CrewAI

Vercel AI SDK critical for TS devs

MCP development

[*] @modelcontextproto... [*] MCP Inspector [HON] Hono [NPM] npm

97M monthly SDK downloads

Codebase parsing

[AST] Tree-sitter [*] tree-sitter-typescript [DB] pgvector [VDB] Qdrant

Stable

Observability

[LS] LangSmith [OBS] Langfuse [*] Vercel AI SDK tracing

Use both: LangSmith for graphs, Langfuse for OSS

Q3 Tools -- Fine-Tuning, DPO, MLOps

Category

Tools

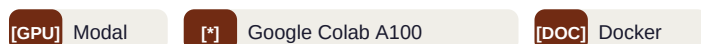
Status

Fine-tuning core



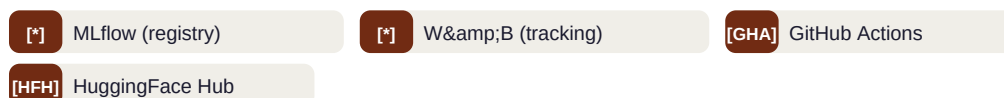
Unsloth = default single-GPU 2026

Compute



Modal replaces Colab for long runs

MLOps



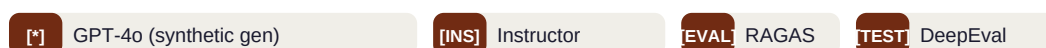
All stable

Inference serving



SGLang 29% faster on H100s

Data + eval



All stable

Q4 Tools -- Portfolio, Cert, Job Search

Category

Tools

Status

Portfolio + demos



Gradio = fastest demo UI 2026

Certification



New cert launched 2024, stable

Job search



All stable

Key Resources and References

One primary resource per topic. Go deep on one rather than skimming ten.

Books

- Chip Huyen -- AI Engineering (2025): The definitive production AI engineering reference. Read Chapters 3 (RAG), 6 (Evaluation), 9 (Fine-tuning) during the matching quarters.
- Chip Huyen -- Designing Machine Learning Systems: System design interview preparation. Chapter 11 is essential.
- Sebastian Raschka -- Build a Large Language Model from Scratch: Understand the internals before you fine-tune them.

Courses

- DeepLearning.AI -- 'Building and Evaluating Advanced RAG' (short course, free): Covers hybrid search, reranking, and eval. Do this in Month 1.
- DeepLearning.AI -- 'LangGraph' (short course, free): Do this in Month 4 before the LangGraph implementation weeks.
- DeepLearning.AI -- 'Finetuning Large Language Models' (short course, free): Do this in Month 7.
- Hugging Face NLP Course (free, huggingface.co/learn): Chapters on fine-tuning and PEFT are excellent supplementary material for Q3.

Papers (skim, don't study deeply)

- QLoRA (Dettmers et al. 2023): Read Sections 1-3 in Month 7. Understand NF4 quantisation and LoRA interaction.
- RAGAS (Es et al. 2023): Read the metric definitions in Section 3 before Month 2.
- DPO (Rafailov et al. 2023): Read Sections 1-3 in Month 8. The math in Section 4 is optional.
- CoBERT (Khattab & Zaharia 2020): Skim Sections 1-3 in Month 1. Understand late interaction at a conceptual level.
- CRAG (Yan et al. 2024): Read before Week 4. Understand the retrieve -> grade -> fallback loop.

Newsletters and blogs

- The Batch (DeepLearning.AI, weekly): industry news, one technical highlight per issue. Subscribe from day 1.
- Sebastian Raschka -- Ahead of AI (Substack): deep technical content on LLMs and fine-tuning. Subscribe before Q3.
- Lilian Weng -- lilianweng.github.io: the best long-form technical deep dives on agents, RAG, and alignment.
- Latent Space podcast (audio): Swyx and Alessio cover the AI engineering space at exactly the right depth. Listen while commuting.
- Eugene Yan -- eugeneyan.com: applied ML, RAG, and evaluation. Very practical, very relevant.

Communities

- Hugging Face Discord -- #fine-tuning and #rag channels: ask questions, see what others are building, share your work.
- LangGraph Discord: the fastest way to get help on LangGraph issues during Q2.
- r/LocalLLaMA (Reddit): fine-tuning tips, model benchmarks, and community Unsloth/Axolotl guides.
- AI Engineer World's Fair (conference, San Francisco, annual): the primary networking event for the AI engineer community. If possible, attend the year-end event.

End-of-Year Checklist

By the end of month 12, you should be able to check every item on this list. This is what 'ready to be a Senior AI Engineer' looks like in 2026.

Production RAG (Q1)

- Can explain the difference between single-vector, hybrid (BM25 + dense), and late-interaction (ColBERT) retrieval
- Can implement a cross-encoder reranking stage and measure NDCG improvement
- Can set up automated eval CI with RAGAS + DeepEval thresholds blocking merge
- Can run adversarial tests with Promptfoo and interpret the results
- Can deploy a RAG service to ECS, Lambda, and a VPS -- and explain the tradeoffs
- Can instrument a production system with Langfuse and OpenTelemetry

AI Agents and MCP (Q2)

- Can build a stateful multi-turn agent in LangGraph with checkpointed memory
- Can implement the supervisor pattern for routing queries across multiple sub-agents
- Can build a TypeScript agent application using Vercel AI SDK with streaming and tool calling
- Can build, test, and publish a TypeScript MCP server on npm
- Can debug MCP tool calls using MCP Inspector
- Can explain the 3-index design (code / rules / domain) and justify the query routing approach

Fine-tuning and MLOps (Q3)

- Can fine-tune a 7B model using QLoRA on a single A100 GPU in under an hour
- Can explain NF4 quantisation, LoRA rank/alpha tradeoffs, and gradient checkpointing
- Can build a DPO preference dataset and run DPO training on top of SFT
- Can set up a full CI/CD training pipeline: git push -> train -> eval -> registry -> serve
- Can serve a fine-tuned model with LoRA adapter via vLLM with the OpenAI-compatible API
- Can run a pre/post RAGAS comparison and explain the delta in plain English

Portfolio and career (Q4)

- 3 live, demoable projects with public GitHub repos, architecture diagrams, and benchmark tables
- Google Cloud Professional Machine Learning Engineer (PME) certification badge on LinkedIn
- 9+ LinkedIn posts published during the year -- at least one with > 1000 impressions
- npm package with > 50 downloads -- the MCP server is real and used
- Can walk through any of the 3 projects in a 5-minute demo and answer deep technical questions
- Can whiteboard a production RAG system design in 10 minutes without preparation

The path was: build something real in Q1, build something harder in Q2, build something you couldn't have built without Q1 and Q2 in Q3, and then show the world in Q4. Every week counts. Every commit compounds.